CSCI 6461 | TOPIC 3

The Hardware/Software Interface & AArch64 Programmer's Model

Architectural state, registers, memory organization, and foundational instruction mechanics

Dr. J. Burns
Computer Systems Architecture — AArch64 Reference Platform

Topic 3: Module Topics

1. **Architectural State & The Hardware Contract**
   - Concrete definition of architectural state vs. microarchitectural structures.
   - The AArch64 programmer's model and processor execution modes.
2. **AArch64 Register File & Naming Conventions**
   - The 31 general-purpose registers: 64-bit (`x0`–`x30`) vs. 32-bit (`w0`–`w30`).
   - Special-purpose registers: Zero register (`xzr`/`wzr`), stack pointer (`sp`), program counter (`pc`).
   - Standard ABI register usage: arguments, return values, caller-saved, and callee-saved registers.
3. **Memory Organization & Endianness**
   - Byte-addressable 64-bit address space, memory alignment rules, and byte ordering.
4. **Core Instruction Classes & Execution Mechanics**
   - Data movement, arithmetic/logical operations, NZCV condition flags, and condition codes.

PART 1

Architectural State & Execution Model

What Is Architectural State?

- **Definition:** The complete set of software-visible hardware storage locations required to represent a program's execution at any discrete point in time.
- If execution is interrupted (context switch, interrupt, exception), saving and restoring this exact state allows program resumption without alteration of behavior.
- **Core Components in AArch64:**
  - General-Purpose Registers (`x0`–`x30`).
  - Program Counter (`pc`) and Stack Pointer (`sp`).
  - Current Program Status Register / Condition Flags (`pstate` / NZCV).
  - System control and configuration registers (e.g., TTBR0_EL1, SCTLR_EL1 for exception/virtualization management).
  - Byte-addressed memory contents.

State Checkpointing

Operating systems actively rely on the strict definition of architectural state to successfully suspend and resume millions of threads per second without computational corruption.

Architectural vs. Microarchitectural State

Architectural State (Visible Contract)

- Defined strictly by the ISA specification.
- Software directly reads/writes this state.
- **Examples:** Registers `x0`–`x30`, `pc`, architectural memory, NZCV flags.

Microarchitectural State (Hidden)

- Internal hardware implementation details.
- Invisible to software correctness.
- **Examples:** Pipeline latches, L1/L2 caches, Branch Target Buffers, rename tables.

The Execution Boundary

Hardware may wildly execute instructions out of order across hidden microarchitectural buffers, but it **must** commit all final mathematical results sequentially to the visible architectural state.

The AArch64 Execution State

- **AArch64:** Pure 64-bit execution state introduced in ARMv8-A.
- **Flat 64-bit Virtual Addressing:** Registers hold 64-bit pointers (architectural address space up to 2^{64} bytes). Fixed 4-byte instruction width remains permanently aligned to 4-byte memory boundaries.
- **Clean-Slate RISC Design:** Eliminates legacy A32 quirks by removing universal conditional execution, removing `ldm`/`stm` loops, and decoupling the Program Counter (`pc`) entirely from the general register file.

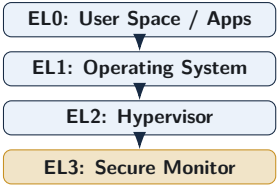
The Legacy-Free Advantage

By abandoning strict backwards compatibility with 32-bit ARM inside the AArch64 execution mode, engineers freed up massive decode logic and instruction-encoding space.

Privilege Levels (Exception Levels)

AArch64 Exception Levels

- **EL0 (Application):** Unprivileged user execution space.
- **EL1 (OS Kernel):** Privileged operating system kernel.
- **EL2 (Hypervisor):** Virtualization layer for guest OSs.
- **EL3 (Secure Monitor):** Firmware and security management.



Course Scope

Our architectural analysis focuses exclusively on the standard general-purpose user-space execution model (**EL0**) and its direct interactions with the OS (**EL1**).

PART 2

AArch64 Register Architecture & ABI

General-Purpose Registers: x0–x30

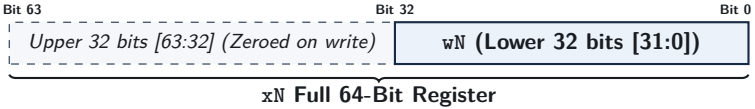
- AArch64 provides **31 general-purpose registers** (x0–x30), each exactly **64 bits wide** (8 bytes).
- **Orthogonal Design:** Any of these registers can serve interchangeably as a source, destination, or base address pointer without restriction.
- **Simplified ISA:** Eliminates the highly dedicated register constraints (e.g., implicit counter registers) found in legacy architectures like x86.

Why Exactly 31 Registers?

RISC instructions allocate precisely 5 bits to encode a register operand ($2^5 = 32$). AArch64 dedicates 31 of these slots to general storage and reserves the 32nd slot for a hardwired Zero Register / Stack Pointer alias.

32-Bit Register Views: w0–w30

- Every 64-bit x register has a corresponding 32-bit view denoted as w0 through w30.
- wN represents the **lower 32 bits (bits 31:0)** of the physical register xN.
- This aliasing enables efficient execution of native 32-bit integer operations (like C int variables) without requiring separate hardware register files.



Zero-Extension Safety Rule

Writing any data to a 32-bit w register **automatically zero-extends** the value, completely wiping out the upper 32 bits of the corresponding x register to clear out potential garbage data.

The Zero Register: `xzr` and `wzr`

- Encoded as register number 31 within most standard ALU instruction contexts.
- **Read Behavior:** Always returns the constant value `0` (64-bit `0` for `xzr`, 32-bit `0` for `wzr`).
- **Write Behavior:** Silently discards any value written to it (serving as a digital bit-bucket).
- **Architectural Benefit:** Eliminates the need to design dedicated zero-loading instructions or comparison formats, vastly simplifying the ISA.

Common Idioms Using the Zero Register

Assembly Instruction	Equivalent Operation	Purpose
`mov x0, xzr`	$x_0 \leftarrow 0$	Clear register x_0 to zero
`subs xzr, x1, x2`	Evaluate $x_1 - x_2 \rightarrow$ flags	Compare x_1 and x_2 (alias: `cmp x1, x2`)
`str xzr, [x1]`	Mem$[x_1] \leftarrow 0$	Zero out a 64-bit memory word

Special-Purpose Registers: `sp`, `pc`, `lr`

Stack Pointer (`sp`)

- Dedicated register holding current stack top address.
- Must strictly maintain **16-byte alignment** when accessing memory in the ABI.
- Arithmetic on `sp` is restricted exclusively to stack allocations/deallocations.

Program Counter (`pc`)

- Holds memory address of executing instruction.
- **Cannot** be read or written directly as a data register (no `mov x0, pc`).
- Accessed indirectly via PC-relative addressing (`adr`, `adrp`).
- Modified via branch and call instructions.

Link Register (`x30` / `lr`)

Holds the direct return address generated by subroutine branch-with-link instructions (`bl`, `blr`). A function returning execution merely writes this value back into the `pc`.

The Calling Convention: Caller- vs. Callee-Saved

Caller-Saved Registers (`x0`–`x17`)

- Primarily used for temporary scratch computations and argument passing.
- The caller **must** save them to the stack before making a subroutine call if their values are needed later.
- The called function (callee) is completely free to overwrite them without restoring.

Callee-Saved Registers (`x19`–`x28`)

- Safely holds long-lived variables across multiple function calls.
- If a called function modifies any of these, it **must save the original value to the stack** and successfully restore it before returning.
- Guarantees protection of caller state.

The Efficiency Trade-off

Balancing caller- and callee-saved registers across the ABI deliberately minimizes unnecessary memory traffic and stack spilling during deeply nested software logic.

AArch64 Register File Overview

Register	Architectural Role	Preserved Across Calls? (ABI)
`x0–x7`	Function arguments & return values	No (Caller-saved / Scratch)
`x8`	Indirect result location (struct return pointer)	No (Caller-saved)
`x9–x15`	Temporary / Scratch registers	No (Caller-saved)
`x16–x17`	Intra-procedure-call temporary registers (IP0, IP1)	No (Caller-saved)
`x18`	Platform register (reserved for OS / TLS)	Platform-dependent
`x19–x28`	Callee-saved registers	**Yes (Callee-saved)**
`x29` (`fp`)	Frame pointer (base of current stack frame)	**Yes (Callee-saved)**
`x30` (`lr`)	Link register (subroutine return address)	No (Caller-saved)
`sp`	Dedicated stack pointer (16-byte aligned)	**Yes (Preserved)**
`xzr`	Dedicated zero register (constant 0, writes discarded)	N/A (Hardware constant)

Process State: `pstate` and Condition Flags

- The processor maintains an internal process state register (`pstate`). The upper 4 bits evaluate the **NZCV Arithmetic Condition Flags**:

Flag	Name	Condition Under Which Flag Is Set to 1
N	**Negative**	Result bit 63 (or bit 31) is 1 (negative signed value)
Z	**Zero**	Computed result is exactly zero
C	**Carry**	Operation generated an unsigned carry-out (or no borrow in sub)
V	**Overflow**	Signed arithmetic overflow occurred (sign bit is invalid)

Selective Flag Updates

Standard ALU instructions (`add`, `sub`) do **not** modify flags. Only instructions with an "`s`" suffix (`adds`, `subs`) or comparison instructions (`cmp`) update NZCV natively.

PART 3

Memory Model, Alignment & Endianness

Memory Organization & Byte Addressing

- Memory is physically organized as a contiguous sequence of **8-bit bytes**.
- Every individual byte is assigned a unique 64-bit architectural address.
- Data operands span multiple consecutive byte addresses scaling by standard powers of two:

Data Type	Size (Bytes)	Size (Bits)	AArch64 Architecture Term
Byte	1	8-bit	Byte (`b`)
Halfword	2	16-bit	Halfword (`h`)
Word	4	32-bit	Word (`w`)
Doubleword	8	64-bit	Doubleword (`x`)
Quadword	16	128-bit	Quadword / Vector element (`q`)

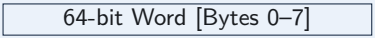
64-Bit Pointer Limitations

While AArch64 uses 64-bit pointers, current physical silicon and page table designs typically

Memory Alignment Rules

- An access to an n-byte operand is **naturally aligned** if its memory address is an exact mathematical multiple of n: Address $(\mathrm{mod}\ n) = 0$.
- **Alignment Requirements:**
  - 32-bit Word access: Address must be a multiple of 4 (Address $(\mathrm{mod}\ 4) = 0$).
  - 64-bit Doubleword access: Address must be a multiple of 8 (Address $(\mathrm{mod}\ 8) = 0$).
  - Instruction fetch: Address must always be a strict multiple of 4.

Aligned Access (Single Transaction)



Unaligned Access (Crosses Boundary)

Alignment Penalties

Unaligned memory accesses can cross cache-line boundaries, requiring multiple memory transactions and severely degrading bandwidth. Strict environments will throw a hardware fault on unaligned access.

Byte Ordering: Little-Endian vs. Big-Endian

Storage of 32-bit Word `0x0a0b0c0d` at Base Address `0x1000`: Demonstrating how multi-byte values are mapped into sequential byte-addressable memory locations.

Little-Endian (AArch64 Default)

- **Least Significant Byte (LSB)** stored at the **lowest** memory address.

Address	Stored Byte
`0x1000`	`0x0d` (LSB)
`0x1001`	`0x0c`
`0x1002`	`0x0b`
`0x1003`	`0x0a` (MSB)

Big-Endian (Network Order)

- **Most Significant Byte (MSB)** stored at the **lowest** memory address.

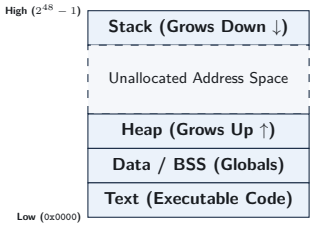
Address	Stored Byte
`0x1000`	`0x0a` (MSB)
`0x1001`	`0x0b`
`0x1002`	`0x0c`
`0x1003`	`0x0d` (LSB)

Network Interoperability

While standard OS environments execute exclusively in **little-endian** mode, raw incoming internet packets adhere strictly to Big-Endian network order, requiring byte-swap instructions (`rev`) to parse

The Process Address Space Layout

- **Text (Code):** Read-only machine instructions.
- **Data / BSS:** Globally accessible and static variables.
- **Heap:** Dynamically allocated memory via `malloc`; grows **upward** toward higher addresses.
- **Stack:** Local function variables and saved caller state; grows **downward** toward lower addresses.



Stack-Heap Collision

The expansive unallocated gap ensures the Heap and Stack operate independently. Unchecked recursion causes the stack to crash into the heap, resulting in a segmentation fault.

PART 4

Foundational Instruction Classes

Instruction Syntax Conventions

- Standard AArch64 assembly strictly follows: `mnemonic dst, src1, src2_or_imm`
- **Three-Operand Format:** Computation safely preserves the source registers.

Assembly Example	RTL Operation	Description
`add x0, x1, x2`	$x_0 \leftarrow x_1 + x_2$	64-bit register addition
`add w0, w1, w2`	$w_0 \leftarrow w_1 + w_2$	32-bit addition (clears upper 32 bits of x_0)
`add x0, x1, #16`	$x_0 \leftarrow x_1 + 16$	Addition with immediate constant

Non-Destructive Architecture

Unlike x86's destructive two-operand syntax (`add eax, ebx` modifies `eax`), AArch64 retains original variables post-computation by routing output to a third register.

Arithmetic Instructions: Addition & Subtraction

Instruction	Operation	Flags Updated?
`add xd, xn, xm`	$x_d \leftarrow x_n + x_m$	No
`adds xd, xn, xm`	$x_d \leftarrow x_n + x_m$	**Yes (Updates NZCV)**
`sub xd, xn, xm`	$x_d \leftarrow x_n - x_m$	No
`subs xd, xn, xm`	$x_d \leftarrow x_n - x_m$	**Yes (Updates NZCV)**
`neg xd, xm`	$x_d \leftarrow 0 - x_m$ (Alias: `sub xd, xzr, xm`)	No
`negs xd, xm`	$x_d \leftarrow 0 - x_m$ (Alias: `subs xd, xzr, xm`)	**Yes**

Immediate Scaling

Immediate values (`#imm12`) span 0 to 4095 and can be natively shifted left by 12 bits in hardware via `lsl #12`.

Logical Operations: Bitwise Manipulation

- Bitwise operations execute concurrently across all 64 parallel bit slices in the ALU.

Mnemonic	Operation	Bitwise Logic
`and xd, xn, xm`	Bitwise AND	$x_d \leftarrow x_n \wedge x_m$
`orr xd, xn, xm`	Bitwise OR	$x_d \leftarrow x_n \vee x_m$
`eor xd, xn, xm`	Bitwise XOR	$x_d \leftarrow x_n \oplus x_m$
`bic xd, xn, xm`	Bit Clear (AND NOT)	$x_d \leftarrow x_n \wedge (\sim x_m)$
`mvn xd, xm`	Bitwise NOT	$x_d \leftarrow \sim x_m$
`ands xd, xn, xm`	Bitwise AND with Flags	$x_d \leftarrow x_n \wedge x_m \rightarrow$ NZCV

Bit Clear (`bic`) Utility

`bic x0, x1, x2` acts as a hardware mask, clearing every bit in x_1 where x_2 is 1.

Shift Operations: Barrel Shifter Integration

- AArch64 hardware encompasses a dedicated **barrel shifter** integrated onto the second operand datapath.
- Shift instructions execute as standalone mathematical operations or fold silently into primary ALU instructions.

Mnemonic	Operation Name	Mathematical Meaning
`lsl xd, xn, #k`	Logical Shift Left	$x_d \leftarrow x_n \ll k$ (Multiplication by 2^k)
`lsr xd, xn, #k`	Logical Shift Right	$x_d \leftarrow x_n \gg k$ (Unsigned division by 2^k, zero-fill)
`asr xd, xn, #k`	Arithmetic Shift Right	$x_d \leftarrow x_n \gg k$ (Signed division by 2^k, sign-fill)
`ror xd, xn, #k`	Rotate Right	Circular right bit rotation

Integrated Shifted Register Example

`add x0, x1, x2, lsl #3` $\implies x_0 \leftarrow x_1 + (x_2 \times 8)$ encoded dynamically as a **single instruction with hardware-shifted operand support** without cycle penalty.

Comparison and Testing Instructions

- Comparison instructions exclusively evaluate operands, update NZCV status flags, and immediately discard the computed mathematical value.

Instruction	Underlying Operation	Flags Updated
`cmp xn, xm`	`subs xzr, xn, xm`	Evaluates $x_n - x_m \rightarrow$ NZCV
`cmn xn, xm`	`adds xzr, xn, xm`	Evaluates $x_n + x_m \rightarrow$ NZCV (Compare Negative)
`tst xn, xm`	`ands xzr, xn, xm`	Evaluates $x_n \wedge x_m \rightarrow$ NZCV (Bit Test)

Flag Interpretation Rules

If $x_n == x_m$, `cmp xn, xm` activates the **Z (Zero)** flag since $x_n - x_m = 0$. ▪ If $x_n < x_m$ (signed), `cmp` aligns flags so that N $\neq$ V (Negative differs from Overflow).

Loading Immediate Constants: `movz`, `movk`, `movn`

- A fixed 32-bit instruction cannot physically fit an arbitrary 64-bit constant.
- AArch64 constructs massive constants iteratively in 16-bit chunks using **wide-immediate instructions**:

Instruction	Operation Name	Semantic Action
`movz xd, imm, lsl s`	Move Wide with Zero	Loads 16-bit constant at shift s; clears rest
`movk xd, imm, lsl s`	Move Wide with Keep	Overwrites 16 bits at shift s; **keeps** other bits intact
`movn xd, imm, lsl s`	Move Wide with NOT	Loads bitwise inverted 16-bit constant directly

Constructing a 32-bit Value (`0x1234abcd`)

```
movz x0, #0xabcd              ⟹ x0 = 0x000000000000abcd   (Lower 16 bits loaded)
movk x0, #0x1234, lsl #16     ⟹ x0 = 0x000000001234abcd   (Bits 31:16 overwritten)
```

Memory Access: Basic Load/Store Architecture

- Memory is accessed universally via explicit load (`ldr`) and store (`str`) mechanisms.
- The register size specifier strictly dictates the physical data transfer width:

Instruction	Register	Transfer Width	Action
`ldr xt, [xn]`	64-bit (`xt`)	8 Bytes (Doubleword)	$x_t \leftarrow \text{Mem}_8[x_n]$
`str xt, [xn]`	64-bit (`xt`)	8 Bytes (Doubleword)	$\text{Mem}_8[x_n] \leftarrow x_t$
`ldr wt, [xn]`	32-bit (`wt`)	4 Bytes (Word)	$w_t \leftarrow \text{Mem}_4[x_n]$ (Zero-extended to x_t)
`str wt, [xn]`	32-bit (`wt`)	4 Bytes (Word)	$\text{Mem}_4[x_n] \leftarrow w_t$
`ldrb wt, [xn]`	8-bit (`wt`)	1 Byte	$w_t \leftarrow \text{Mem}_1[x_n]$ (Zero-extended)
`strb wt, [xn]`	8-bit (`wt`)	1 Byte	$\text{Mem}_1[x_n] \leftarrow w_t[7:0]$

Pointer Referencing

The base register (x_n) holds the 64-bit memory pointer, explicitly enclosed in square brackets `[xn]` to signify architectural indirection.

Addressing Modes: Base with Immediate Offset

Mechanism (EA = x_n + offset)

- Instruction: `ldr xt, [xn, #offset]`
- Accesses memory physically at x_n + offset.
- **No side effects:** Base register x_n remains completely unchanged post-execution.

Code Examples

`ldr x1, [x0]`	Load $[x_0]$
`ldr x2, [x0, #8]`	Load $[x_0 + 8]$
`str x2, [x0, #16]`	Store $[x_0 + 16]$

Hardware Offset Range

Standard immediate offsets are structured as unsigned 12-bit scaled values. For 64-bit doublewords, the hardware scales this value up by 8, yielding a maximum effective offset of $+32760$ bytes.

Advanced Addressing: Pre- and Post-Indexed

Pre-Indexed (Update Before Access)

```
ldr xt, [xn, #imm]!
```

1. Update base: $x_n \leftarrow x_n + \text{imm}$.
2. Access memory at updated address: $x_t \leftarrow \text{Mem}[x_n]$.

Note the exclamation mark (`!`) syntax.

Post-Indexed (Update After Access)

```
ldr xt, [xn], #imm
```

1. Access memory at current address: $x_t \leftarrow \text{Mem}[x_n]$.
2. Update base: $x_n \leftarrow x_n + \text{imm}$.

Base update happens *after* the load.

Efficiency Impact

Pointer tracking naturally embeds into memory instructions, thereby eliminating standalone add overhead when rapidly traversing structural arrays or popping execution stack elements.

Loading and Storing Pairs (`ldp` and `stp`)

- **Core Purpose:** Transfer two completely adjacent 64-bit registers in a single instruction pipeline.
- **Primary Use Case (Stack Management):**
  - Combines two discrete register movements into one atomic 128-bit memory operation.
  - Efficiently saves and restores critical function linkage (Frame Pointer x_{29} and Link Register x_{30}) securely during active function calls.
- **Performance Impact:** Issues a unified 128-bit memory bus transaction, fundamentally doubling data transfer throughput compared to split single operations.

Stack Frame Construction Example

```
// Push FP (x29) and LR (x30) to stack, pre-decrementing SP by 16 bytes
stp x29, x30, [sp, #-16]!
```

Control Flow: Unconditional Branches

- Control flow instructions structurally redirect program execution by coercing the Program Counter (pc).

Instruction	Name / Syntax	Operational Behavior
`b label`	Direct Branch	pc $\leftarrow$ label (PC-relative range: ± 128 MB)
`br xn`	Indirect Branch	pc $\leftarrow x_n$ (Jump to address held in register x_n)
`bl label`	Branch with Link	$x_{30} \leftarrow$ pc $+ 4$; pc $\leftarrow$ label (Subroutine call)
`blr xn`	Indirect Call	$x_{30} \leftarrow$ pc $+ 4$; pc $\leftarrow x_n$ (Function pointer call)
`ret`	Subroutine Return	pc $\leftarrow x_{30}$ (Default return via Link Register)

PC-Relative Addressing

Direct branch offsets natively encode as signed binary displacements relative to the current pc. This structure mathematically guarantees compiled code executes perfectly completely irrespective of loaded memory position.

Conditional Branches & Condition Codes

Syntax: `b.cond label`

Executes immediate branch execution jumping to `label` only if the specified NZCV condition configuration matches evaluating criteria to true.

Condition Code	Meaning	Flag Condition	Comparison Context
`b.eq`	Equal / Zero	$Z == 1$	Signed / Unsigned
`b.ne`	Not Equal / Non-Zero	$Z == 0$	Signed / Unsigned
`b.lt`	Signed Less Than	$N \neq V$	Signed
`b.le`	Signed Less Than or Equal	$(Z == 1) \vee (N \neq V)$	Signed
`b.gt`	Signed Greater Than	$(Z == 0) \wedge (N == V)$	Signed
`b.ge`	Signed Greater or Equal	$N == V$	Signed
`b.lo`	Unsigned Lower ($<$)	$C == 0$	Unsigned
`b.hi`	Unsigned Higher ($>$)	$(C == 1) \wedge (Z == 0)$	Unsigned

Control Flow Idiom: If-Else Translation

High-Level C Code

```
if (a == b) {
    c = c + 1;
} else {
    c = c - 1;
}
```

AArch64 Assembly Mapping

```
// Assume: x0=a, x1=b, x2=c
cmp x0, x1        // Compare a and b
b.ne .Lelse       // If a != b, jump to else
add x2, x2, #1    // Then-body: c = c + 1
b .Lend           // Skip else-body
.Lelse:
sub x2, x2, #1    // Else-body: c = c - 1
.Lend:
```

Assembly Inversion Pattern

High-level conditions frequently invert their evaluation test logically in compiled assembly code, executing the branch actively to jump completely *around* the positive then-body.

Control Flow Idiom: While-Loop Translation

High-Level C Loop

```
// Compute sum of array
uint64_t i = 0;
while (i < n) {
    sum += arr[i];
    i++;
}
```

AArch64 Assembly Translation

```
// x0=arr, x1=n, x2=i, x3=sum
mov x2, xzr                  // i = 0
mov x3, xzr                  // sum = 0
.Lloop:
cmp x2, x1                   // Test i < n
b.ge .Ldone                  // Exit if i >= n
ldr x4, [x0, x2, lsl #3]     // Load arr[i]
add x3, x3, x4               // sum += arr[i]
add x2, x2, #1               // i++
b .Lloop                     // Repeat loop
.Ldone:
```

Loop Unrolling Context

Executing two branching instructions per iteration severely bottlenecks simple evaluation

Conditional Select Instructions: Eliminating Branches

- Branch instructions natively introduce deep pipeline branch misprediction risks.
- AArch64 deliberately mitigates pipeline flushes by deploying **conditional select instructions** that strictly choose between loaded operands without branching:

Instruction	Semantic Operation	C Ternary Equivalent
`csel xd, xn, xm, cond`	$x_d \leftarrow \text{cond} \,?\, x_n : x_m$	`d = cond ? n : m;`
`cinc xd, xn, cond`	$x_d \leftarrow \text{cond} \,?\, (x_n + 1) : x_n$	`d = n + (cond ? 1 : 0);`
`cset xd, cond`	$x_d \leftarrow \text{cond} \,?\, 1 : 0$	`d = (cond) ? 1 : 0;`

Example: Finding Maximum of Two Values ($\max(x_1, x_2)$)

```
cmp x1, x2              // Compare x1 and x2
csel x0, x1, x2, gt     // If x1 > x2, x0 = x1; else x0 = x2 (0 branch penalties!)
```

Review and Self-Test Questions

Topic 3: Comprehensive Summary

- **Architectural State:** Precise hardware-software contract fundamentally defined by general registers (`x0`–`x30`), flags (NZCV), stack pointer (`sp`), program counter (`pc`), and byte-addressed memory allocations.
- **Register Structure:** 31 orthogonal 64-bit registers (`x0`–`x30`) structurally alias 32-bit blocks (`w0`–`w30`); writing to a `w` register safely and strictly clears the upper 32 bits natively.
- **ABI Rules:** `x0`–`x7` handle execution arguments and returns (caller-saved); `x19`–`x28` securely hold local variables across calls (callee-saved).
- **Load/Store Operations:** Arithmetic executes exclusively in registers; memory interactions operate strictly via `ldr`/`str` offset, indexed, or unified paired (`ldp`/`stp`) modes.
- **Control Flow:** Branching explicitly evaluates logic via NZCV flags updated dynamically by `adds`/`subs`/`cmp`; utilizing conditional select (`csel`) mathematically

Self-Test Questions: State & Register Mechanics

1. **Architectural vs. Microarchitectural State:** Why are internal CPU branch prediction tables and physical register rename structures classified uniquely as microarchitectural state rather than architectural state?
2. **32-bit Register Write Semantics:** Suppose physical register `x5` initially contains hexadecimal `0xffffffff_ffffffff`. If the target instruction `mov w5, #0x1234` is successfully executed, what exact 64-bit value natively resides in `x5` post-execution?
3. **Zero Register Application:** Why does the AArch64 specification provide a dedicated zero register (`xzr`/`wzr`) functionally instead of permanently dedicating a general-purpose register (like `x0`) to continuously hold zero?
4. **Calling Convention Responsibility:** If user function `foo()` directly calls function `bar()`, and `foo()` actively needs to preserve computation values across the call in registers `x3` and `x20`, which function assumes technical responsibility for securing each register to the stack?

Self-Test Questions: Memory & Instruction Mechanics

1. **Memory Addressing Differences:** Explain the exact mechanical difference in physical execution behavior and final register state configurations between:
   - `ldr x1, [x0, #16]`
   - `ldr x1, [x0, #16]!`
   - `ldr x1, [x0], #16`
2. **Endianness Evaluation:** The 32-bit word `0xdeadbeef` is physically stored at memory address `0x2000` on a little-endian AArch64 system. What is the exact mathematical byte value stored at address `0x2000`? At address `0x2003`?
3. **Branch Elimination via `csel`:** Convert the following C code snippet directly into an equivalent branchless AArch64 assembly sequence safely deploying `cmp` and `csel`:

```
int64_t result = (val1 <= val2) ?  (val1 + 10) :  (val2 - 5);
```

4. **Condition Flag Verification:** If `x0` correctly contains integer 5 and `x1` contains 10, meticulously evaluate and describe which NZCV condition flags are actively asserted following instruction `cmp x0, x1`.

Looking Ahead to Topic 4: Assembly Programming

Topic 4 Preview (Arithmetic, Logic, & Control Flow)

Moving from architectural structure to complex functional application.

- **ALU & Barrel Shifting:** Mastering intensive arithmetic operations natively coupled with zero-penalty second-operand hardware shifts.
- **Bitwise Logic & Field Extraction:** Efficiently manipulating interior bit arrays structurally utilizing masks, bit clears, and unrolled `ubfx` commands.
- **Condition Flags & Comparisons:** Leveraging native NZCV bits dynamically and choosing strategically between signed vs. unsigned branch conditions.
- **Control Flow & Branchless Selection:** Functionally translating loops and dense `if-else` programming blocks, utilizing `csel` structurally to eliminate costly branch penalties entirely.